TYPESCRIPT MASTER CHEATSHEET

Readable edition • Fundamentals to advanced concepts • SDET examples

LEARN concepts **PRACTICE** code **APPLY** in automation

This version uses safer page breaks, smaller code samples, wider usable margins, and more readable code blocks to prevent content from being clipped or pushed off the page.

Reference-inspired by the official TypeScript cheatsheets, Devhints, Coddy, and DevSheets. Examples are newly written for this guide.

How to use this guide

Read the explanation, run the example, then change an input to test your understanding. TypeScript catches many errors before execution; JavaScript is what actually runs.

Marker	Meaning
Rule	Syntax or type-system rule to remember.
Runtime	Behavior of emitted JavaScript.
SDET use	Where the concept helps in UI, API, database, or functional automation.
Pitfall	Common mistake or misleading assumption.

Setup

```
npm init -y
npm install --save-dev typescript
npx tsc --init
npx tsc --noEmit
```

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "noUncheckedIndexedAccess": true
  }
}
```

REMEMBER Choose module settings for your runtime (Node, browser bundler, or test runner). Strict mode is recommended for new projects.

Contents

01 Console, comments, and syntax
02 Variables, scope, hoisting, and memory
03 Data types, annotations, and type safety
04 Operators and evaluation
05 Conditions and loops
06 Named functions
07 Anonymous functions
08 Arrow functions
09 Function choice in development/testing
10 Callbacks and async callbacks
11 Callback patterns
12 Function overloading
13 Overloading vs overriding
14 Overloading examples
15 Arrays and methods
16 Array iteration
17 forEach, map, filter, reduce, sum, some, every
18 Strings
19 Objects and key-value pairs
20 Object access and methods
21 Type aliases/intersections/classes
22 Classes and properties
23 Constructors and methods

24 Object independence
25 Static members
26 Shared members and overloads
27 Constructor/method overloads
28 Class internals and memory
29 Inheritance basics
30 Inheritance types/constructors
31 Inheritance internals
32 super/access modifiers/polymorphism
33 Inheritance in automation
34 Interfaces basics
35 Optional/readonly/method signatures
36 Interface inheritance
37 Classes implementing interfaces
38 Modules
39 SDET patterns
40 Common mistakes
41 Practice set
42 Answer key
43 Quick syntax recap
44 References

01 • Console, comments, and basic syntax

console.log()

Writes values to the host console: browser DevTools or the Node.js terminal.

```
const user = "Maya";
const attempts = 3;
console.log("User:", user);
console.log({ user, attempts });
```

Comments

```
 // Single-line comment
 /*
  Multi-line comment
  explains a non-obvious decision.
 */
 /** JSDoc-style documentation */
function retryCount(): number { return 3; }
```

• Comment why a decision exists rather than restating the code.

• Use structured logging instead of scattered console.log() calls in production systems.

02 • Variables, scope, redeclaration, hoisting, and memory

Declaration and initialization

```
let score: number;  // declared; runtime value is undefined
score = 10;         // assignment
let retries = 2;    // declaration + initialization
const baseUrl = "https://example.test";
var legacyFlag = true; // avoid in modern code
```

Keyword	Scope	Redeclare same scope?	Reassign?	Hoisting
var	Function/global	Yes	Yes	Hoisted; initialized undefined
let	Block	No	Yes	TDZ before declaration
const	Block	No	No	TDZ before declaration

Scope and redeclaration

```
function demo() {
  var functionScoped = "function";
  if (true) {
    let blockScoped = "block";
    const fixed = 42;
    console.log(blockScoped, fixed);
  }
  console.log(functionScoped);
  // console.log(blockScoped); // Error
}

let count = 1;
count = 2; // reassignment is fine
// let count = 3; // redeclaration error
```

const and object mutation

```
const config = { retries: 2 };
config.retries = 4; // allowed
// config = {};     // error: reassigning binding
```

Hoisting

```
console.log(flag); // undefined
var flag = true;

// let mode = "safe";
// Accessing mode before declaration is a TDZ error.
```

Memory model

A variable holds a value or a reference to an object. Exact physical stack/heap placement is engine-dependent; think in terms of bindings, object identity, reachability, and garbage collection.

```
const first = { status: "new" };
const second = first;
second.status = "done";
console.log(first.status); // "done": same object
```

REMEMBER Use const by default, let when reassignment is needed, and avoid var unless maintaining legacy code.

03 • Data types, annotations, and type safety

JavaScript is dynamically typed. TypeScript checks types before execution, then generally erases type annotations when emitting JavaScript.

Type	Example	Common use
string	let name: string = "Asha";	Text, labels, URLs
number	let total: number = 99.5;	Counts and numeric values
boolean	let passed = true;	Flags
null/undefined	let result: string | null = null;	Missing/unknown values
Array	const ids: number[] = [1,2];	Collections
Tuple	const pair: [string, number] = ["A",1];	Fixed positions
Union	let id: string | number = 10;	One of several shapes
void	function log(): void {}	No useful return value
never	function fail(): never { throw Error(); }	Cannot complete normally
unknown	let payload: unknown;	Untrusted input; narrow first
any	let unsafe: any;	Disables checks; avoid

Inference, annotation, and narrowing

```
let quantity = 4; // inferred number
let label: string = "Ready";

type UserId = string | number;
function formatId(id: UserId): string {
  if (typeof id === "number") return `#${id}`;
  return id.toUpperCase();
}
```

Interface and type alias

```
interface User {
  readonly id: number;
  name: string;
  email?: string;
}
type Status = "passed" | "failed" | "skipped";
type Result =
  | { ok: true; data: User }
  | { ok: false; error: string };
```

Five real-world examples

• Form model: age is number; email is string.

• API result: union separates success data from error information.

• Test status: literal union prevents invalid labels.

• Framework configuration: type baseURL, timeout, and retry count.

• Database row: model nullable fields explicitly, e.g. string | null.

REMEMBER Type annotations do not validate external JSON. Treat input as unknown and validate it at runtime.

04 • Operators and evaluation

Category	Operators	Example
Arithmetic	+ - * / % **	7 % 3 is 1
Assignment	= += -= *= /=	count += 1
Comparison	< <= > >=	age >= 18
Equality	== != === !==	status === 'ready'
Logical	&& || !	loggedIn && hasRole
Nullish/optional	?? ?.	user.name ?? 'Guest'
Increment	++	index++
Ternary	condition ? a : b	ok ? 'PASS' : 'FAIL'
Type operators	typeof, instanceof, in, keyof	typeof x === 'string'

Strict equality

```
console.log(5 == "5");  // true: coercion
console.log(5 === "5"); // false: different types
console.log(null == undefined);  // true
console.log(null === undefined); // false
```

Pre/post increment

```
let a = 4;
const oldValue = a++; // oldValue=4, a=5
let b = 4;
const newValue = ++b; // b=5, newValue=5
```

Short-circuit and nullish coalescing

```
const admin = false;
const allowed = admin && deleteRecord(); // function not called
const display = suppliedName ?? "Guest"; // null/undefined only
const label = suppliedName || "Guest";   // all falsy values
```

• Prefer ===/!==. Use parentheses to clarify precedence. Avoid side effects in complex expressions.

• Use ?? if 0, false, and empty string are valid values.

Type operators

```
typeof value === "string"; // runtime
item instanceof Date;      // runtime
"email" in user;           // runtime
type UserKeys = keyof User; // compile-time only
```

05 • Conditional statements and loops

```
if (response.ok) {
  console.log("Success");
} else if (response.status === 404) {
```

```
  console.log("Not found");
} else {
  console.log("Unexpected");
}

switch (status) {
  case "passed": console.log("green"); break;
  case "failed": console.log("red"); break;
  default: console.log("neutral");
}

for (let i = 0; i < 3; i++) console.log(i);
while (attempts < 3) { attempts++; }
do { console.log("once minimum"); } while (false);

for (const key in record) console.log(key); // keys
for (const value of values) console.log(value); // values
```

• break exits; continue skips to the next iteration.

• Use for...of for array values. for...in is usually not the right array loop.

• In Playwright, prefer auto-waiting locator assertions over fixed sleeps.

06 • Named functions

Named function declarations are reusable and are hoisted as declarations by JavaScript.

```
function multiply(a: number, b: number): number {
  return a * b;
}
console.log(multiply(3, 4)); // 12
```

Optional, default, rest, union parameters

```
function greet(name: string, title?: string): string {
  return title ? `Hello ${title} ${name}` : `Hello ${name}`;
}
function timeoutMessage(ms = 5000): string {
  return `Timeout: ${ms}`;
}
function sumAll(...values: number[]): number {
  return values.reduce((total, n) => total + n, 0);
}
function printId(id: string | number): void {
  console.log(id);
}
```

• Optional parameters use ? and follow required parameters.

• Default parameters apply when an argument is omitted or undefined.

• Rest parameters collect remaining arguments into an array.

07 • Anonymous functions

An anonymous function has no declared name; it is often assigned to a variable or passed as a callback.

```
const divide = function (a: number, b: number): number {
  if (b === 0) throw new Error("Cannot divide by zero");
  return a / b;
};
[1, 2, 3].forEach(function (n) { console.log(n); });

type Comparator<T> = (a: T, b: T) => number;
const byLength: Comparator<string> = function (a, b) {
  return a.length - b.length;
};
```

• Use named helpers for business actions; anonymous functions are handy for short local behavior.

08 • Arrow functions

```
const square = (n: number): number => n * n;
const greet = (name: string) => `Hello ${name}`;
const makeUser = (name: string) => ({ name, active: true });
```

Arrow functions have lexical this: they capture it from the surrounding context. Regular functions get this from how they are called. Arrow functions cannot be used with new.

```
class Counter {
  value = 0;
  start() {
    setInterval(() => { this.value++; }, 1000);
  }
}
```

09 • Function choice in development and testing

Kind	Strength	Best fit
Named declaration	Clear identity; hoisted declaration	Reusable business actions and test helpers
Anonymous function	Local inline behavior	Callback passed once
Arrow function	Concise; lexical this	map/filter predicates, event handlers

```
async function login(page: Page, username: string, password: string) {
  await page.getByLabel("Username").fill(username);
  await page.getByLabel("Password").fill(password);
  await page.getByRole("button", { name: "Sign in" }).click();
}
const isSuccess = (status: number) => status >= 200 && status < 300;
const activeUsers = users.filter(user => user.active);
```

• Web development: event handlers and transformations.

• Unit tests: setup helpers and pure assertions.

• Functional/UI automation: login(), createOrder(), assertToast().

• API testing: response mapping, schema checks, retry logic.

• DB testing: row mappers and predicates for expected records.

REMEMBER Use names that communicate business intent; avoid deeply nested anonymous callbacks.

10 • Callbacks and asynchronous callbacks

A callback is a function passed to another function for later or conditional execution.

```
function runCheck(
  check: () => boolean,
  onResult: (ok: boolean) => void
) {
  onResult(check());
}
runCheck(() => 2 + 2 === 4, ok => console.log(ok));
```

• The receiving function controls when and how often the callback runs.

• Callbacks can close over outer variables.

• Async callbacks may run after the current call stack completes.

```
setTimeout(() => console.log("Later"), 100);

async function loadUser(id: number): Promise<User> {
  const response = await fetch(`/api/users/${id}`);
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json() as Promise<User>;
}
```

The cast in the example is not runtime validation. Validate untrusted JSON before relying on its shape.

Avoid callback pyramids

```
const user = await getUser();
const orders = await getOrders(user.id);
const report = await buildReport(orders);
```

11 • Callback patterns

```
type RetryCheck = (attempt: number) => Promise<boolean>;

async function retryUntil(check: RetryCheck, max = 3): Promise<boolean> {
  for (let attempt = 1; attempt <= max; attempt++) {
    if (await check(attempt)) return true;
  }
  return false;
}
```

• Use callbacks for predicates, mapping, assertions, and event completion.

• For sequential async tasks, use for...of with await. For safe independent tasks, Promise.all may run them concurrently.

12 • Function overloading

Overloads expose multiple call signatures while one implementation handles the behavior. Callers are checked against overload signatures, not the implementation signature.

```
function format(value: string): string;
function format(value: number): string;
function format(value: string | number): string {
  return typeof value === "number" ? value.toFixed(2) : value.trim();
}
```

```
function createLabel(name: string): string;
function createLabel(name: string, prefix: string): string;
function createLabel(name: string, prefix = "User"): string {
  return `${prefix}: ${name}`;
}
```

• Prefer a union when one signature is clear. Use overloads when input forms produce meaningfully different output types.

13 • Overloading vs overriding

Concept	Meaning
Overloading	Multiple TypeScript call signatures; one implementation
Overriding	Subclass replaces an inherited method implementation

```
class Parent {
  describe(): string { return "parent"; }
}
class Child extends Parent {
  override describe(): string { return "child"; }
}
```

REMEMBER Java supports multiple method bodies by parameter list; TypeScript overloads use declarations plus one runtime implementation.

14 • Overloading examples

```
function parseValue(value: string): string[];
function parseValue(value: number): number;
function parseValue(value: string | number): string[] | number {
  return typeof value === "number" ? value : value.split(",");
}
const parts = parseValue("a,b"); // string[]
const amount = parseValue(10);   // number
```

• Overload signatures must come before the implementation.

• Use typeof, Array.isArray, or discriminant properties to choose the runtime path.

• Test every advertised call signature.

15 • Arrays: declaration and methods

```
const names: string[] = ["Asha", "Ben"];
const scores: Array<number> = [90, 82];
const point: [number, number] = [10, 20];
const ids: readonly number[] = [1, 2, 3];
console.log(names[0], names.length);
```

Method	Purpose	Mutates? / result
push/pop	Add/remove at end	Mutates
shift/unshift	Remove/add at start	Mutates
slice	Copy range	No mutation
splice	Insert/remove range	Mutates
includes	Membership check	Boolean
find	First matching item	Item or undefined
sort	Sort array	Mutates
join	Join to string	Returns string

• Array indices start at zero. Out-of-range access can be undefined.

• sort() mutates; use a copy or toSorted() where supported if immutability matters.

16 • Array iteration and functions

```
const values = [10, 20, 30];
for (let i = 0; i < values.length; i++) console.log(i, values[i]);
for (const key in values) console.log(key); // keys
for (const value of values) console.log(value); // values

function average(numbers: number[]): number {
  if (numbers.length === 0) throw new Error("No values");
  return numbers.reduce((sum, n) => sum + n, 0) / numbers.length;
}
```

• Functions can accept arrays as parameters and return typed arrays, e.g. function failed(items: TestCase[]): TestCase[].

17 • forEach, map, filter, reduce, sum, every, some

Examples

```
items.forEach(item => console.log(item)); // side effects
const names = users.map(user => user.name); // transform
const active = users.filter(user => user.active); // select
const total = [2, 3, 4].reduce((sum, n) => sum + n, 0); // 9
const allPassed = results.every(r => r.status === "passed");
const hasFailure = results.some(r => r.status === "failed");
```

Method	Returns	Use when
forEach	undefined	Side effects
map	New array	Transform every item
filter	New array	Keep matches
reduce	Accumulator	Aggregate/group/index

Method	Returns	Use when
sum	Not built-in	Write helper with reduce
every	boolean	All values match
some	boolean	At least one matches

• Do not use forEach(async ...) expecting it to await each task.

```
for (const user of users) {
  await updateUser(user); // sequential
}
await Promise.all(users.map(updateUser)); // concurrent, if safe
```

REMEMBER Use readable chains for simple transformations. Avoid unnecessary repeated passes over very large collections.

18 • Strings

```
const text = "  TypeScript Master  ";
text.length;
text.charAt(2); text[2]; text.indexOf("Script");
text.includes("Type");
text.slice(2, 12); text.substring(2, 12);
text.replace("Master", "Guide");
text.split(" "); text.trim();
text.toUpperCase(); text.toLowerCase();

const user = "Asha";
const greeting = `Hello, ${user}!`;
const multi = `First line
Second line`;
```

Strings are immutable: methods return new strings. They do not alter the original string.

```
function isNonEmpty(value: string): boolean {
  return value.trim().length > 0;
}
function looksLikeEmail(value: string): boolean {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value);
}
```

A simple regex is only a format check, not proof an email address exists.

19 • Objects and key-value pairs

```
const user = { id: 101, name: "Asha", active: true };
console.log(user.name);
console.log(user["id"]);

type Credentials = { username: string; password: string };
const credentials: Credentials = {
  username: "qa.user",
  password: process.env.TEST_PASSWORD ?? ""
};
```

• Object literals are the commonest way to create objects. Other options include new Date(), class constructors, and Object.create().

REMEMBER Never hard-code real credentials or API keys into source code or reports.

20 • Object methods and property access

```
const cart = {
  items: 2,
  total: 250,
  summary() { return `${this.items} items; total ${this.total}`; }
};
cart.items = 3;
cart["total"] = 300;
```

- Dot notation suits known identifier-like property names.
- Bracket notation supports dynamic keys and names like record["display-name"].

```
const field = "email";
const record: Record<string, string> = { email: "qa@example.test" };
console.log(record[field]);
```

21 • Inline types, aliases, intersections, and classes

```
function printUser(user: { id: number; name: string }): void {
  console.log(user.id, user.name);
}
type Identified = { id: number };
type Named = { name: string };
type Person = Identified & Named;
const person: Person = { id: 1, name: "Asha" };
```

Feature	Best for
Inline type	One-off small shape
type alias	Unions, intersections, tuples, named shapes
interface	Object contract that may be extended
class	Runtime instances with state and behavior

22 • Classes: properties and constructors

```
class TestConfig {
  readonly projectName: string;
  retries: number;
  browser?: string;

  constructor(projectName: string, retries = 2) {
    this.projectName = projectName;
    this.retries = retries;
  }
}
```

Modifier	Meaning
public	Accessible from anywhere; default
private	Restricted by TypeScript to declaring class
protected	Class and subclasses
readonly	Cannot reassign through typed access after initialization

REMEMBER readonly is a compile-time restriction; it does not freeze an object at runtime.

23 • Constructors and instance methods

```
class ApiClient {
  constructor(private readonly baseUrl: string, private timeoutMs = 5000) {}
  setTimeout(ms: number): void {
    if (ms <= 0) throw new Error("Timeout must be positive");
    this.timeoutMs = ms;
  }
  endpoint(path: string): string {
    return `${this.baseUrl}/${path}`;
  }
}
```

Constructor parameter properties (public/private/protected/readonly) declare and initialize members. Optional values should be handled explicitly.

```
class Profile {
  constructor(public name: string, public nickname?: string) {}
```

```
  displayName(): string { return this.nickname ?? this.name; }
}
```

24 • Object independence

```
class Box { constructor(public size: number) {} }
const first = new Box(10);
const second = new Box(20);
first.size = 15;
console.log(first.size);  // 15
console.log(second.size); // 20

const alias = first;
alias.size = 99;
console.log(first.size); // 99: same instance
```

25 • Static members

```
class Environment {
  static readonly defaultTimeout = 5000;
  static current = "test";
  static isProduction(): boolean {
    return Environment.current === "production";
  }
}
```

• Static members belong to the class, not each instance.

• Useful for constants and factories; avoid mutable static test state in parallel suites.

26 • Shared properties and overloads

```
class Reporter {
  static completed = 0;
  constructor(public name: string) {}
  complete() { Reporter.completed++; }

  format(value: string): string;
  format(value: number): string;
  format(value: string | number): string {
    return typeof value === "number" ? value.toFixed(0) : value.toUpperCase();
  }
}
```

Static counters are shared and can create test-order or parallelism problems.

27 • Constructor and method overloads

```
class SearchQuery {
  term: string;
  page: number;
  constructor(term: string);
  constructor(term: string, page: number);
  constructor(term: string, page = 1) {
    this.term = term;
    this.page = page;
  }
}
```

• TypeScript overloads have multiple declarations and one implementation.

• The implementation must correctly handle every advertised signature.

28 • Class internals and memory

```
class UserSession {
  static activeCount = 0;
  constructor(public userId: string) { UserSession.activeCount++; }
  describe() { return `Session for ${this.userId}`; }
```

```
}
const session = new UserSession("U-1");
```

Member	Mental model	Shared?
Instance field	Property on each instance	No
Prototype method	Found through prototype lookup	Method function shared
Static field	Property on class constructor	Yes
Closure variable	Captured lexical binding	Closures may share it

REMEMBER Memory details vary by engine. Garbage collection reclaims unreachable objects.

29 • Inheritance fundamentals

```
class BasePage {
  constructor(protected pageName: string) {}
  title(): string { return this.pageName; }
}
class LoginPage extends BasePage {
  constructor() { super("Login"); }
}
```

Language	High-level model
Java	Class-based; one direct superclass; interfaces
JavaScript	Prototype-based; class syntax uses prototypes
TypeScript	JavaScript runtime inheritance plus static checking

30 • Inheritance types and constructors

```
class Animal {
  constructor(public name: string) {}
  speak(): string { return "sound"; }
}
class Dog extends Animal {
  constructor(name: string, public breed: string) { super(name); }
  override speak(): string { return "woof"; }
}
interface CanRun { run(): void }
interface CanSwim { swim(): void }
class WorkingDog extends Dog implements CanRun, CanSwim {
  run() {}
  swim() {}
}
```

• TypeScript supports single class inheritance and multiple interface implementation, not multiple base classes.

• Call super() before using this in a derived constructor.

31 • Inheritance internal flow

```
new Dog("Rex", "Collie")
  -> Dog constructor starts
  -> super("Rex") initializes Animal state
  -> Dog initializes breed
  -> instance is returned
  -> dog.speak() resolves to Dog.prototype.speak
```

Method lookup checks the instance and then its prototype chain. A child override is found before the parent method.

• Prefer composition when a class merely uses a capability rather than being a true subtype.

32 • super, access modifiers, and polymorphism

```
class ApiPage {
```

```
  constructor(protected baseUrl: string) {}
  openPath(path: string): string { return `${this.baseUrl}/${path}`; }
}
class AdminPage extends ApiPage {
  constructor(url: string, private section: string) { super(url); }
  override openPath(path: string): string {
    return super.openPath(`admin/${this.section}/${path}`);
  }
}
```

• super calls the parent constructor or parent implementation.

• TypeScript private is a type-checking restriction; ECMAScript #private fields enforce privacy at runtime.

33 • Practical inheritance scenarios

```
abstract class BaseTest {
  abstract run(): Promise<void>;
  protected logStart(name: string): void {
    console.log(`START ${name}`);
  }
}
class LoginTest extends BaseTest {
  async run(): Promise<void> {
    this.logStart("LoginTest");
    // arrange -> act -> assert
  }
}
```

• A base class should contain genuinely common lifecycle behavior, not every helper.

• Ask: is-a relationship? stable parent contract? could composition be simpler?

34 • Interfaces fundamentals

```
interface ApiResponse {
  status: number;
  ok: boolean;
  message?: string;
}
const response: ApiResponse = { status: 200, ok: true };
```

Interfaces describe object shape and are erased from emitted JavaScript. TypeScript is structurally typed: compatible shape is usually what matters.

```
interface HasId { id: number }
const row = { id: 5, name: "Invoice" };
const identified: HasId = row; // structurally compatible
```

35 • Optional, readonly, and method signatures

```
interface TestUser {
  readonly id: string;
  name: string;
  email?: string;
  deactivate(reason: string): boolean;
}
```

• Optional properties may be absent; reading one may yield undefined.

• readonly restricts assignment through the typed reference, not runtime mutation.

• Index signatures should use a meaningful value type.

36 • Interface inheritance and type relationships

```
interface Identifiable { id: string }
interface Timestamped { createdAt: Date }
interface Order extends Identifiable, Timestamped { total: number }
type AuditedOrder = Order & { auditId: string };
```

Feature	interface	type alias
Object shape	Yes	Yes
Extends/implements	Yes	Object-shaped aliases can be implemented
Union	Not directly	Yes
Intersection	Can extend compatible interfaces	Yes
Declaration merging	Yes	No duplicate alias merging

37 • Classes implementing interfaces

```
interface Logger { log(message: string): void }
class ConsoleLogger implements Logger {
  log(message: string): void { console.log(message); }
}
function reportFailure(logger: Logger, message: string) {
  logger.log(message);
}
```

• Interfaces support replaceable implementations and test doubles.

• Do not use instanceof with an interface; interfaces do not exist at runtime.

• Use runtime checks or a schema validator for external data.

38 • Modules, imports, exports, runtime

```
 // user.ts
export interface User { id: string; name: string }
export function displayName(user: User): string {
  return user.name.trim();
}

// app.ts
import { displayName } from "./user.js";
import type { User } from "./user.js";

export const DEFAULT_TIMEOUT = 5000;
export default class ApiClient {}

// Named import
import { DEFAULT_TIMEOUT } from "./utils.js";
// Default import
import ApiClient from "./ApiClient.js";
```

• Type-only imports are erased when used only for types.

• Module path conventions depend on Node ESM, CommonJS, and bundlers.

39 • SDET patterns

Typed API result

```
type ApiResult<T> =
  | { ok: true; status: number; data: T }
  | { ok: false; status: number; error: string };

function assertSuccess<T>(result: ApiResult<T>): T {
  if (!result.ok) throw new Error(result.error);
  return result.data;
}
```

Typed test data and runtime guard

```
interface LoginCase {
  name: string;
  username: string;
  password: string;
  expected: "success" | "invalid-credentials" | "locked";
}
function isRecord(x: unknown): x is Record<string, unknown> {
  return typeof x === "object" && x !== null && !Array.isArray(x);
}
```

Testing area	Useful features
UI/web	Page objects, typed fixtures, state unions
Functional	Named actions and explicit test case models
API	unknown input, guards, async/await, response types
Database	Row types, nullable columns, result mappers
Unit testing	Pure functions, interfaces, test doubles
Cross-platform	Interfaces and adapters for browser/device differences

40 • Common mistakes

Mistake	Fix
Using any everywhere	Prefer unknown and narrow
Assuming TS validates API JSON	Validate at runtime
Using == habitually	Prefer ===
Thinking const freezes objects	It only prevents rebinding
forEach with async callbacks	Use for...of + await or Promise.all
for...in for array values	Use for...of
Assuming sum() exists	Use reduce or a helper
Interface at runtime	Use runtime guards instead
Overusing inheritance	Consider composition
Casting away errors	Narrow/validate first

41 • Practice set

- 1. Create a const array, mutate it, and explain why allowed.
- 2. Compare var/let/const scope in an if block.
- 3. Write a string | number function with narrowing.
- 4. Predict x and y for let x=5; const y=x++.
- 5. Write required, optional, and default parameters.
- 6. Use map to select names and filter active users.
- 7. Sum [4,5,6] with reduce and explain the initial value.
- 8. Define an interface with readonly id and optional email.
- 9. Extend a base page and call super().
- 10. Implement a Logger interface twice.
- 11. Write retryUntil(check, attempts) using async/await.

• 12. Export a type and import it with import type.

42 • Answer key

```
1. const prevents rebinding, not array mutation.
2. var is function-scoped; let/const are block-scoped.
3. Narrow with typeof value === "string"/"number".
4. x becomes 6; y gets old value 5.
5. Example: function f(a: string, b?: string, c = 1) {}.
6. users.map(u => u.name); users.filter(u => u.active).
7. Result 15; initial 0 defines the starting accumulator.
8. interface User { readonly id: string; email?: string }
9. Call super() before using this.
10. Each implementation provides log(message: string): void.
11. Await each check and stop on success or exhausted attempts.
12. import type { User } from "./types.js";
```

43 • Quick syntax recap

```
const fixed = 10;
let changing: number = 0;

type ID = string | number;
interface User { id: ID; name: string; email?: string }

function add(a: number, b: number): number { return a + b; }
const square = (n: number) => n * n;

const nums: number[] = [1, 2, 3];
const doubled = nums.map(n => n * 2);
const evens = nums.filter(n => n % 2 === 0);
const total = nums.reduce((sum, n) => sum + n, 0);

function show(x: string | number) {
  return typeof x === "string" ? x.toUpperCase() : x.toFixed(0);
}
```

44 • References and further learning

Official TypeScript Cheat Sheets
https://www.typescriptlang.org/cheatsheets/

Official TypeScript Handbook
https://www.typescriptlang.org/docs/handbook/

Devhints TypeScript Cheatsheet
https://devhints.io/typescript

Coddy TypeScript Cheat Sheet
https://coddy.tech/cheat-sheets/typescript

DevSheets TypeScript PDF
https://devsheets.io/pdfs/typescript-cheat-sheet.pdf

TypeScript Playground
https://www.typescriptlang.org/play

REMEMBER This is an original study guide, not a reproduction of the linked resources. Check examples in your installed TypeScript version and project configuration.